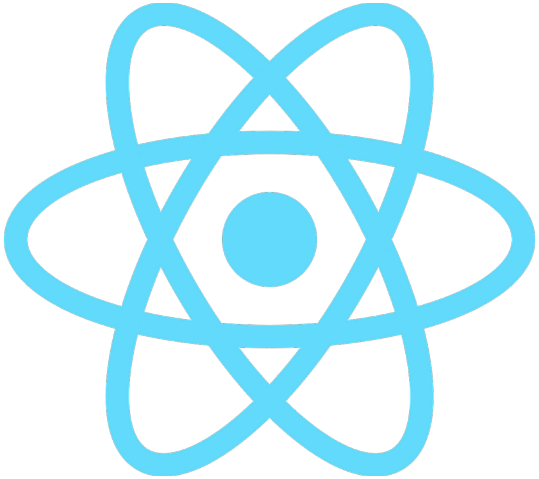
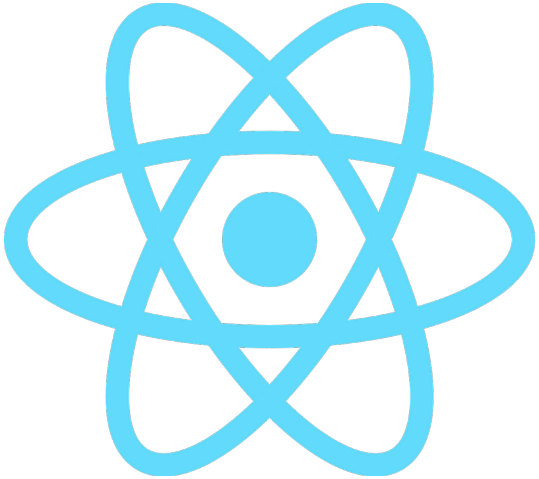


React

Connecting to the world
— data, navigation and deployment

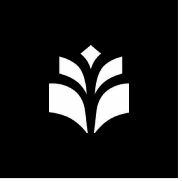
UEF// University of Eastern Finland

Course materials originally by Sonsoles López-Pernas, adapted by Maria Kuismin



React

Student is able to
analyze user interface requirements and design reusable, component-based web applications using React



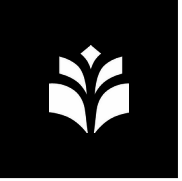
Contents

Fetch in React
- How does a React app retrieve and send data to a server?
- How are loading states and errors handled?
- How are API keys kept safe?

Debugging React applications
- How do you read and fix common React errors?
- How do you debug fetch calls with the Network tab?

SPA and Routing
- What is a Single Page Application?
- How is navigation implemented with react-router?
- How should a larger React application be organized?

Production deployment
- How is a React app built for production?
- How is a React app deployed to GitHub Pages?



Fetch in React

- How does a React app retrieve and send data to a server?
- How are loading states and errors handled?
- How are API keys kept safe?

Debugging React applications

- How do you read and fix common React errors?
- How do you debug fetch calls with the Network tab?

SPA and Routing

- What is a Single Page Application?
- How is navigation implemented with react-router?
- How should a larger React application be organized?

Production deployment

- How is a React app built for production?
- How is a React app deployed to GitHub Pages?

useEffect + fetch — where and why

In vanilla JS: fetch anywhere — click handler, page load
In React: fetch belongs inside useEffect

Why? The component function runs on every render. Fetch directly in component body leads to infinite loop: fetch → setProducts(data) → re-render → fetch again → ...

useEffect with empty dependency array breaks the loop: runs exactly once, when component first appears.

```
useEffect(() => {
    // fetch goes here — inside useEffect
    // not directly in the component body
}, []);
```

Two patterns for async fetch inside useEffect

useEffect cannot be async directly: useEffect(async () => { ... }) ← avoid this

async function defined inside useEffect:

```
useEffect(() => {
    const getData = async () => {
        const res = await fetch("https://...");
        const data = await res.json();
        setProducts(data);
    };
    getData(); ← defined and called immediately
}, []);
```

async function defined at component level:

```
const getData = async () => {
    const res = await fetch("https://...");
    const data = await res.json();
    setProducts(data);
};

useEffect(() => {
    getData();   ← called here, defined above
}, []);
```

Both work. Pattern A: fetch logic stays close to where it is used.
Pattern B: getData can also be called elsewhere (e.g., on button click to refresh data).

Example: fetching data with useEffect

```
function PostList() {
  const [posts, setPosts] = useState([]);    // empty array, not null, .map() requires array
  const getData = async () => {              //  Pattern B: defined here
    const res = await fetch(                 //  can be called elsewhere too
      "https://api.jsonbin.io/bin_id",
      { headers: { "X-Access-Key": import.meta.env.VITE_API_KEY } }
    );
    const data = await res.json();
    setPosts(data.record);   // jsonbin wraps data in .record
  };

  useEffect(() => {
    getData();                               // called once on mount
  }, []);

  return (
    <div>
      {posts.map(post => (
        <PostCard key={post.id} title={post.title} /> // key required for lists
      ))}
    </div>
  );
}
// Note: no loading state or error handling yet
```

Loading state and error handling

```
function PostList() {
  const [posts, setPosts] = useState([]);
  const [isLoading, setIsLoading] = useState(true);    // loading state
  const [error, setError] = useState(null);    // error state

  useEffect(() => {
    const getData = async () => {
      try {
        const res = await fetch('https://api.example.com/posts');
        if (!res.ok) {
          throw new Error(`HTTP error: ${res.status}`);  // catch bad status
        }
        const data = await res.json();
        setPosts(data);
      } catch (err) {
        setError(err.message);  // store error
      } finally {
        setIsLoading(false);    // always runs
      }
    };
    getData();
  }, []);

  if (isLoading) return <p>Loading...</p>;    //  show while loading
  if (error) return <p>Error: {error}</p>;    //  show if failed
  return posts.map(post => <PostCard key={post.id} title={post.title} />);
}
```

GET vs. POST in React

GET — fetch data when component loads:

```
useEffect(() => {
    getData();   ← runs once on mount
}, []);
```

POST — send data when user submits a form:

```
function handleSubmit(event) {
    event.preventDefault();   ← prevent page reload
    await fetch('/api/posts', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(newPost)
    });
    setPosts([...posts, newPost]);   ← update local state
}

<form onSubmit={handleSubmit}>     ← React event handler
```

Env variables — keeping API keys safe

API keys must not be committed to GitHub, anyone can read your source code

1. Create .env file in project root:

```
VITE_API_KEY=your-api-key-here
VITE_BIN_ID=your-bin-id-here
```

2. Add .env to .gitignore:

`.env` ← already there in Vite projects

3. Use in your code:

```
const apiKey = import.meta.env.VITE_API_KEY;
```

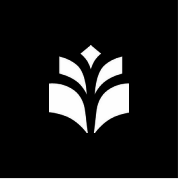
Note: in Vite, all env variables must start with VITE_ otherwise they are not exposed to the browser

⚠️ .env keeps your key out of Git history, but not out of the browser's developer tools when the app is built.

Using jsonbin.io as backend for demo

Features demonstrated in Kažigram
- → GET: fetch posts when app loads
- → PUT: save posts when form submitted or post deleted (replaces entire posts array, not individual resources)
- → Loading spinner while fetching
- → Error notification if fetch fails
- → API key in .env file





Fetch in React

- How does a React app retrieve and send data to a server?
- How are loading states and errors handled?
- How are API keys kept safe?

Debugging React applications

- How do you read and fix common React errors?
- How do you debug fetch calls with the Network tab?

SPA and Routing

- What is a Single Page Application?
- How is navigation implemented with react-router?
- How should a larger React application be organized?

Production deployment

- How is a React app built for production?
- How is a React app deployed to GitHub Pages?

Common React errors

"Cannot read properties of undefined (reading 'title')" → a prop is missing or undefined
check: was the prop passed from the parent?
fix: use optional chaining props.post?.title or a default value: post = {}

"Each child in a list should have a unique key" → .map() is missing a key prop
fix: <PostCard key={post.id} />

"Too many re-renders" → setState called during render, not in an event handler
check: is setState called directly in JSX or at component top level?
fix: move setState inside onClick or useEffect

"X is not defined" / blank screen → missing export default or wrong import path
check: does the filename match the import exactly? import PostCard from './PostCard'

JSX: "Adjacent JSX elements must be wrapped in an enclosing tag" → return contains multiple elements without a wrapper, fix: wrap in <> </> or a div

Network tab for API debugging

DevTools (F12) → Network (shows all requests made by the browser)
→ Filter by "Fetch/XHR" to see only API calls

Status 200 → OK, request succeeded; Status 404 → URL is wrong — check your endpoint; Status 401 → Unauthorized — check your API key; Status 500 → Server error — problem on the API side; CORS error → API does not allow requests from your origin, check: does the API support browser requests?

Click any request to inspect:
Headers → request URL, method, API key
Preview → parsed JSON response
Response → raw response text

When fetch returns nothing:
1. Open Network tab, 2. Find your fetch request, 3. Check the status code,
4. Click Preview → read the error message, 5. Compare the URL to the API documentation

CORS error (Cross-Origin Resource Sharing)

The browser blocked fetch request because the API did not give permission for browser-to-API requests. This is a browser security feature — not a code error.

CORS error appears as: "Access to fetch at 'https://...' from origin 'http://localhost:5173' has been blocked by CORS policy"

1. Check if the API supports browser requests at all → read the API docs (some only work server-to-server)

2. Check if you are missing a required header → jsonbin.io requires headers: { 'X-Master-Key': import.meta.env.VITE_API_KEY } → other APIs may require Authorization: Bearer token

3. If you are developing locally and the API should work → check if you are using the correct endpoint URL → check if HTTPS vs HTTP is causing the issue

CORS cannot be fixed by changing your React code alone. The permission must come from API server.

Debugging — a practical checklist

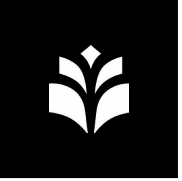
Step 1: Read the error message in the console (F12 → Console) - shows which component and line

Step 2: Check React Developer Tools → Components tab → find the component
- check props and state — are they what you expect?

Step 3: If the problem is data from a server → Network tab → find the fetch request
- check status code and response

Step 4: Add console.log strategically
console.log("posts:", posts); ← check state value
console.log("props:", props); ← check what component received

Step 5: Isolate the problem → comment out parts of the component → simplify to the minimum that still shows the bug



Fetch in React

- How does a React app retrieve and send data to a server?
- How are loading states and errors handled?
- How are API keys kept safe?

Debugging React applications

- How do you read and fix common React errors?
- How do you debug fetch calls with the Network tab?

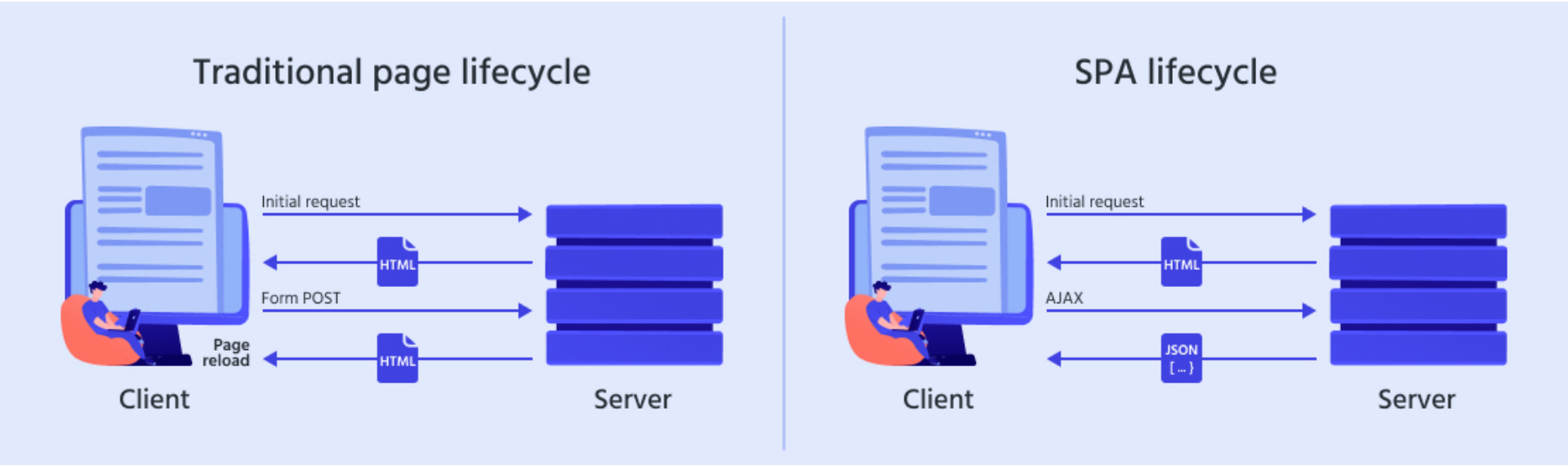
SPA and Routing

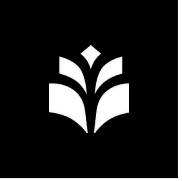
- What is a Single Page Application?
- How is navigation implemented with react-router?
- How should a larger React application be organized?

Production deployment

- How is a React app built for production?
- How is a React app deployed to GitHub Pages?

Single page applications



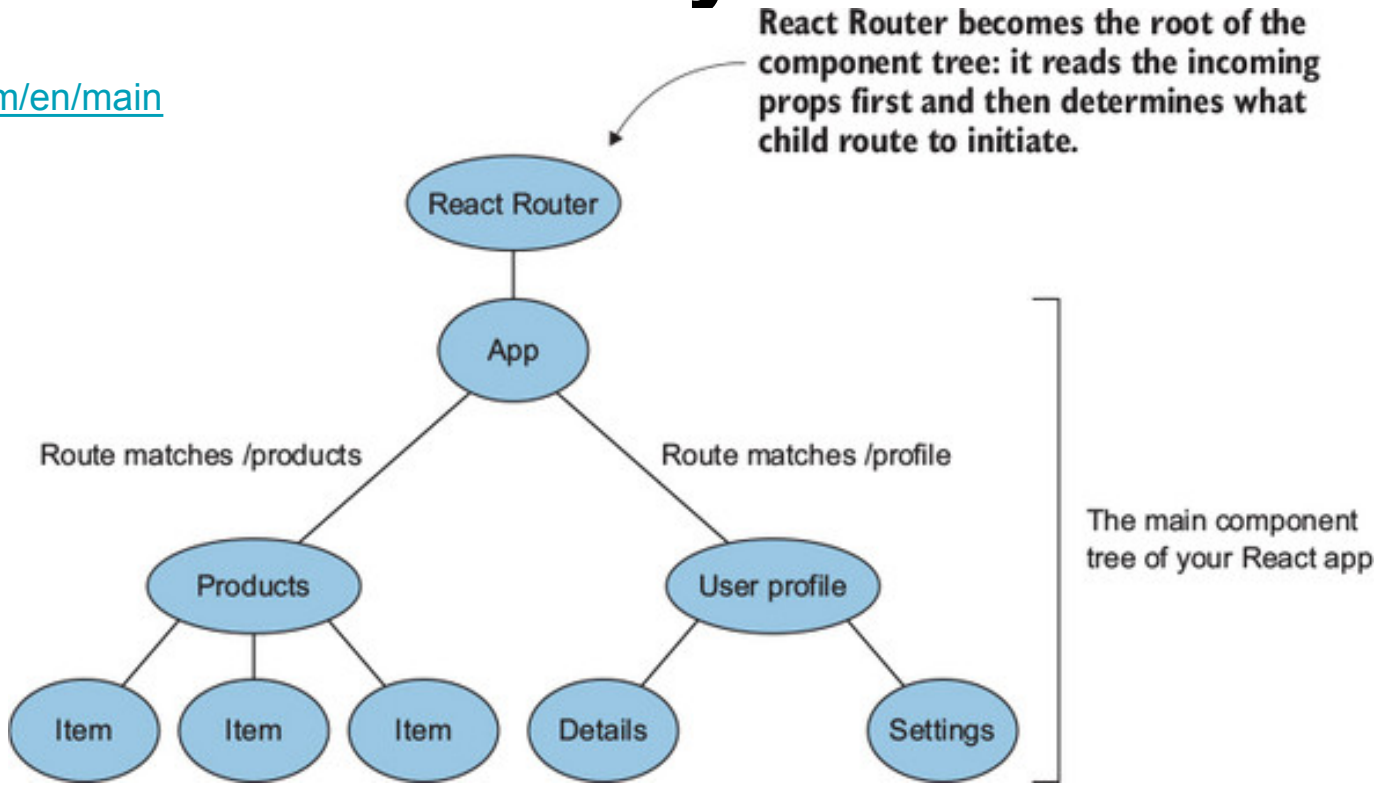


Multipage experience with routing

- routing allows users to have a multi-page experience in SPAs while staying within one HTML document
- client side routing allows app to update the URL from a link click without making another request for another document from the server. Instead, app can immediately render some new UI and make data requests with fetch to update the page with new information.
- enables faster user experiences because the browser doesn't need to request an entirely new document or re-evaluate CSS and JS assets for the next page

The react-router library

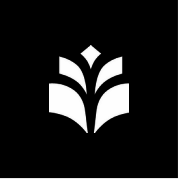
https://reactrouter.com/en/main



Router type: BrowserRouter

BrowserRouter

- **Use case**: This is the most commonly used router for **modern web applications**.
- **How it works**: It uses the **HTML5 history API** (pushState, popState) to keep your UI in sync with the URL.
- **Advantages**:
  - Great for SPAs that run on a web server with proper configuration.
  - No hash (#) in the URL.
- **When to use**: Choose BrowserRouter when your app will be hosted on a web server that can handle client-side routing. This router requires that your server be configured to redirect all non-asset requests to your index.html file.



Router type: HashRouter

HashRouter

- **Use case**: Useful for web applications hosted on servers that **cannot support client-side routing** (e.g., GitHub Pages) or environments where server configurations are more restricted.
- **How it works**: It uses the **URL hash (#)** to simulate routing. The portion of the URL after the # is not sent to the server, so the server doesn't need to know about your app's routing.
- **Advantages**:
  - No server configuration is needed.
  - Works well with older environments or places where you can't modify the server to support history APIs.
- **Disadvantages**: The URL looks messier due to the # (e.g., http://example.com/#/about).
- **When to use**: Ideal for static sites or environments where you can't control the server to handle client-side routing.

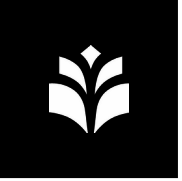
Router type: StaticRouter, MemoryRouter

Other types...

StaticRouter: This router is typically used in **server-side rendering (SSR)** environments, where the routes are determined on the server side and the rendered HTML is sent to the client. Example: next.js.

MemoryRouter: Typically used in **non-browser environments**, such as mobile apps (e.g., with React Native), testing environments, or JavaScript-only apps.

- Off-topic: React can be used to develop native mobile applications (Android and iOS) using React Native (https://reactnative.dev/)



Step 1: Install react-router

```
npm install react-router
```

This is to install the necessary packages

Step 2: Set up the router in main.jsx

Client side routing is enabled by creating a Router and linking/ submitting to pages with Link and <Form>.

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router";
import App from "./app";

const root = document.getElementById("root");

ReactDOM.createRoot(root).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>,
);
```

Step 3: Define routes in App.jsx

```
import { Routes, Route } from "react-router";
import Home from "./pages/Home";
import Posts from "./pages/Posts";
import About from "./pages/About";

function App() {
  return (
    <div>
      <Navbar />  ← Navbar stays outside Routes so it shows on every page
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/posts" element={<Posts />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </div>
  );
}
```

Step 4: Add navigation links Navbar.jsx

```jsx
import { Link } from "react-router";

function Navbar() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/posts">Posts</Link>
      <Link to="/about">About</Link>
    </nav>
  );
}
```

Link updates the URL without reloading the page
<a href> would reload the entire page, breaking SPA

Conditioning routes (e.g. authentication)

```
{isAuthenticated ?

        <Navigate  to="/posts" replace={true} /> :

        <Home setAuthenticated={setAuthenticated} />}
```

If the user is already authenticated, show them the /posts route, otherwise send them to the homepage

Example: Adding router to Kažigram

```jsx
// App.jsx
import { HashRouter, Routes, Route } from 'react-
router'
- -
return (
    <HashRouter>
    <div>
      <NavBar />
      <main>
        <Routes>
          <Route path="/" element={ <Feed
posts={posts} onDelete={handleDelete}/> } />
          <Route path="/new" element={ <NewPostForm
onAddPost={handleAddPost}/>  } />
        </Routes>
      </main>

      <Footer />

    </div>
    </HashRouter>
  )
```

```jsx
// NavBar.jsx
import { Link } from 'react-router'
<Nav>
 <Nav.Link as={Link}
to="/">{t('feed')}</Nav.Link>
 <Nav.Link as={Link}
to="/new">{t('newPost')}</Nav.Link>
</Nav>
```

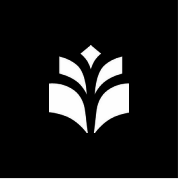
Organizing a larger React app

As your app grows, all files in src/ becomes hard to manage. Recommended structure:

```
src/
   components/          ← reusable components
      PostCard.jsx
      Feed.jsx
      PostForm.jsx
      Header.jsx
   pages/               ← one file per route
      Feed.jsx          ← route: "/"
      NewPost.jsx       ← route: "/new"
   App.jsx              ← routes defined here
   main.jsx             ← entry point
```

pages/ → one component per route

components/ → everything used in multiple pages

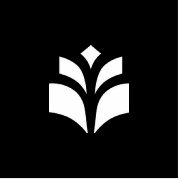


Fetch in React
- How does a React app retrieve and send data to a server?
- How are loading states and errors handled?
- How are API keys kept safe?

Debugging React applications
- How do you read and fix common React errors?
- How do you debug fetch calls with the Network tab?

SPA and Routing
- What is a Single Page Application?
- How is navigation implemented with react-router?
- How should a larger React application be organized?

Production deployment
- How is a React app built for production?
- How is a React app deployed to GitHub Pages?

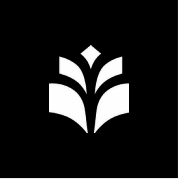


From development to production

When developing, we run `npm run dev` and that starts a development server that transpiles our JSX code and hot-reloads the browser with our latest changes.
… but what happens when we finish developing?
We want static files: HTML, CSS, JS + multimedia ← building

When run the build process (for example, by executing `npm run build` in a React project created with vite), the dev code — including JSX and other assets — is compiled, optimized, and bundled into static files (HTML, CSS, and JavaScript).

These files can then be deployed to a web server and served efficiently to users without requiring a development environment.



Why use React if output is vanilla JS?

Same idea as other developer tools:

- Bootstrap → just CSS in the end
- TypeScript → just JavaScript in the end
- React → just JavaScript in the end

They make development faster (less code to write), more structured (reusable components), more maintainable (clearer code) and less error-prone (state management handled for us)

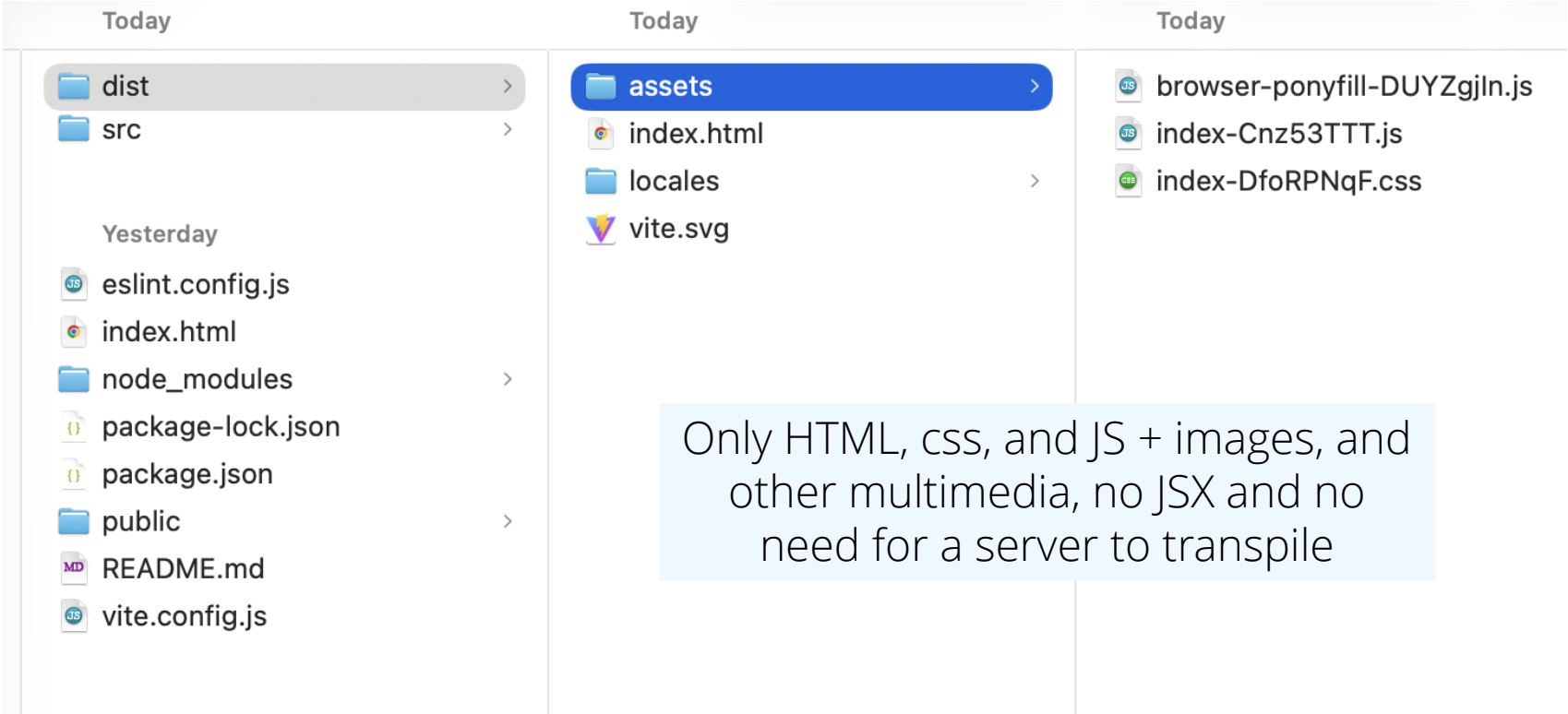
Running the build

```
➜  omdb-react 2 2 git:(main) ✗ npm run build

> omdb-react@0.0.0 build
> vite build

vite v7.1.9 building for production...
✓ 381 modules transformed.
dist/index.html                          0.54 kB │ gzip:  0.33 kB
dist/assets/index-DfoRPNqF.css           0.12 kB │ gzip:  0.12 kB
dist/assets/browser-ponyfill-DUYZgjIn.js   10.30 kB │ gzip:  3.52 kB
dist/assets/index-Cnz53TTT.js            305.49 kB │ gzip: 97.69 kB
✓ built in 632ms
➜  omdb-react 2 2 git:(main) ✗
```

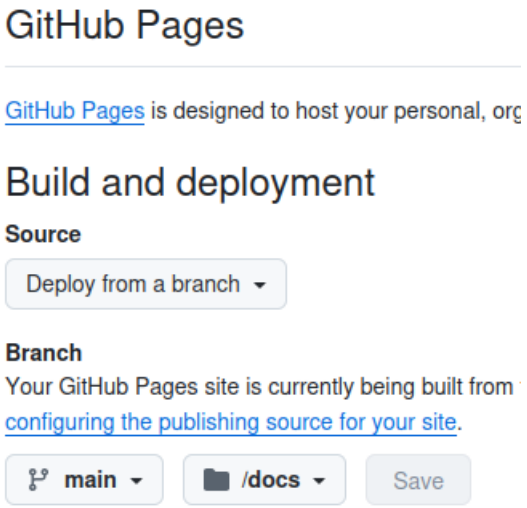
Inside the build folder



Only HTML, css, and JS + images, and other multimedia, no JSX and no need for a server to transpile

Deploying to GitHub Pages

- Before deploying with routing: make sure using HashRouter, not BrowserRouter. → BrowserRouter requires server configuration that GitHub Pages does not support
- Add `base: "/name-of-our-repo/"` to vite.config.js
- Run `npm run build`
- Rename dist folder to docs
- Set up your repo to deploy Github pages from main branch docs folder: Settings → Pages → Branch: main → Folder: /docs → Save



How does it all fit together

SETUP CHAIN

Node.js — runtime environment that runs JavaScript outside the browser

↓ (comes with npm)

npm — package manager that installs libraries (React, Vite, etc.)

↓ (installs)

Vite — development server with hot reloading + production build tool

↓ (uses Babel internally)

React + Babel

DEVELOPMENT CHAIN

WE WRITE React components in JSX (.jsx files)

↓

Vite serves the file

↓

Babel transpiles JSX → vanilla JavaScript

↓

BROWSER RUNS vanilla JavaScript

DEPLOYMENT CHAIN

WE WROTE React components in JSX (.jsx files)

↓

npm run build (one time)

↓

Vite + Babel transpile everything once

↓

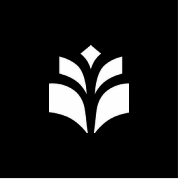
Static files generated: HTML, CSS, JS + assets

↓

Upload to server

↓

BROWSER RUNS vanilla JavaScript (no Vite, no Babel needed)



Summary

- React app retrieves data using fetch inside useEffect with [] and sends data using fetch with method POST in form submit handlers
- Loading states and errors are handled with isLoading and error state variables using try/catch/finally around the fetch call
- API keys are kept safe in a .env file using import.meta.env.VITE_KEY, never commit .env
- Common React errors e.g. undefined props, missing key in .map(), too many re-renders
- Fetch calls are debugged in DevTools Network tab — check status codes
- Single Page Application serves one HTML file — the URL updates without page reload
- Nav is implemented with react-router: wrap App in HashRouter, define Routes in App.jsx
- Larger apps are organized with src/components/ for reusable components and src/pages/ for one component per route
- React app is built with npm run build — dist/ contains only HTML, CSS and JS
- GitHub Pages: use HashRouter, build, rename dist to docs, configure repo settings